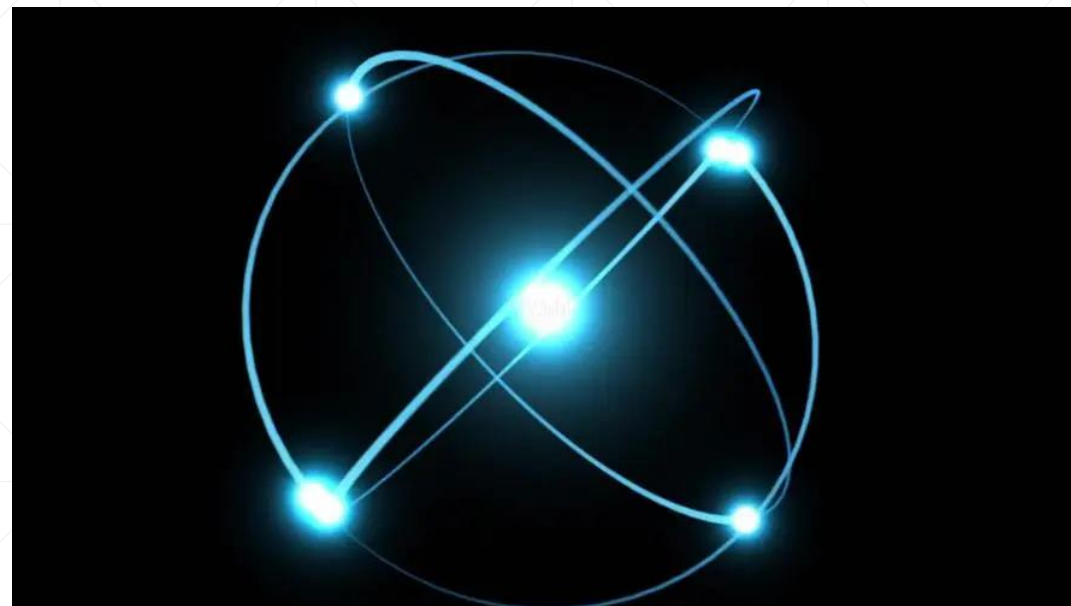


“深入多线程内部机理”之一



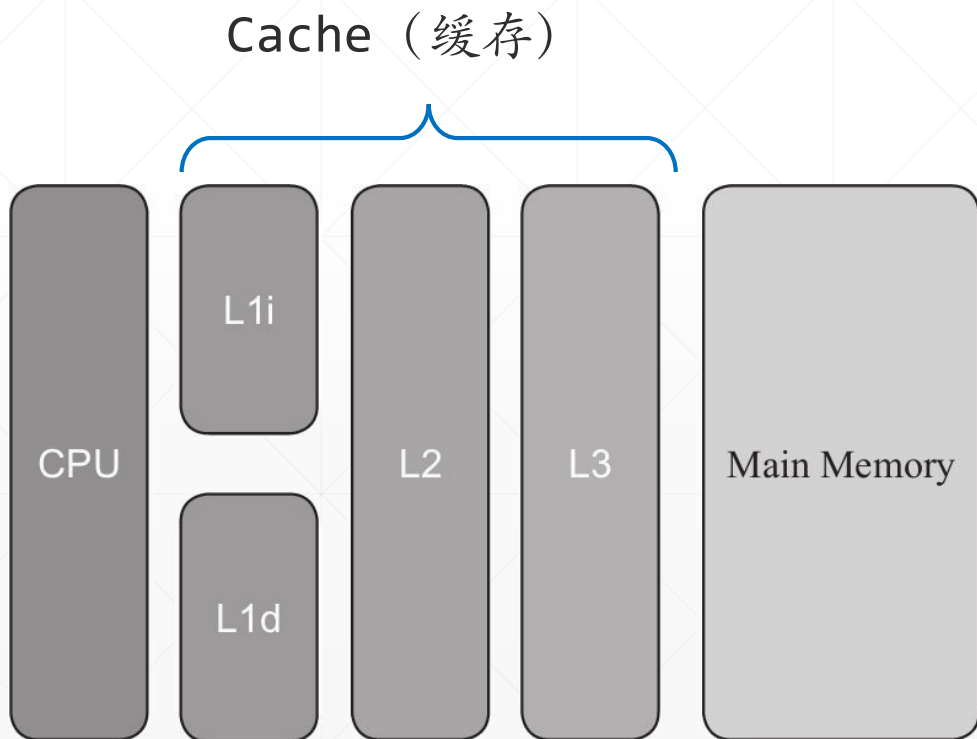
学一点计算机运行原理

北京理工大学计算机学院
金旭亮

计算机中的分级缓存设计

CPU的运行速度与访问主内存的速度，差异非常大。

为了解决两者速度严重不匹配的问题，在CPU与内存之间，添加了高速缓存。

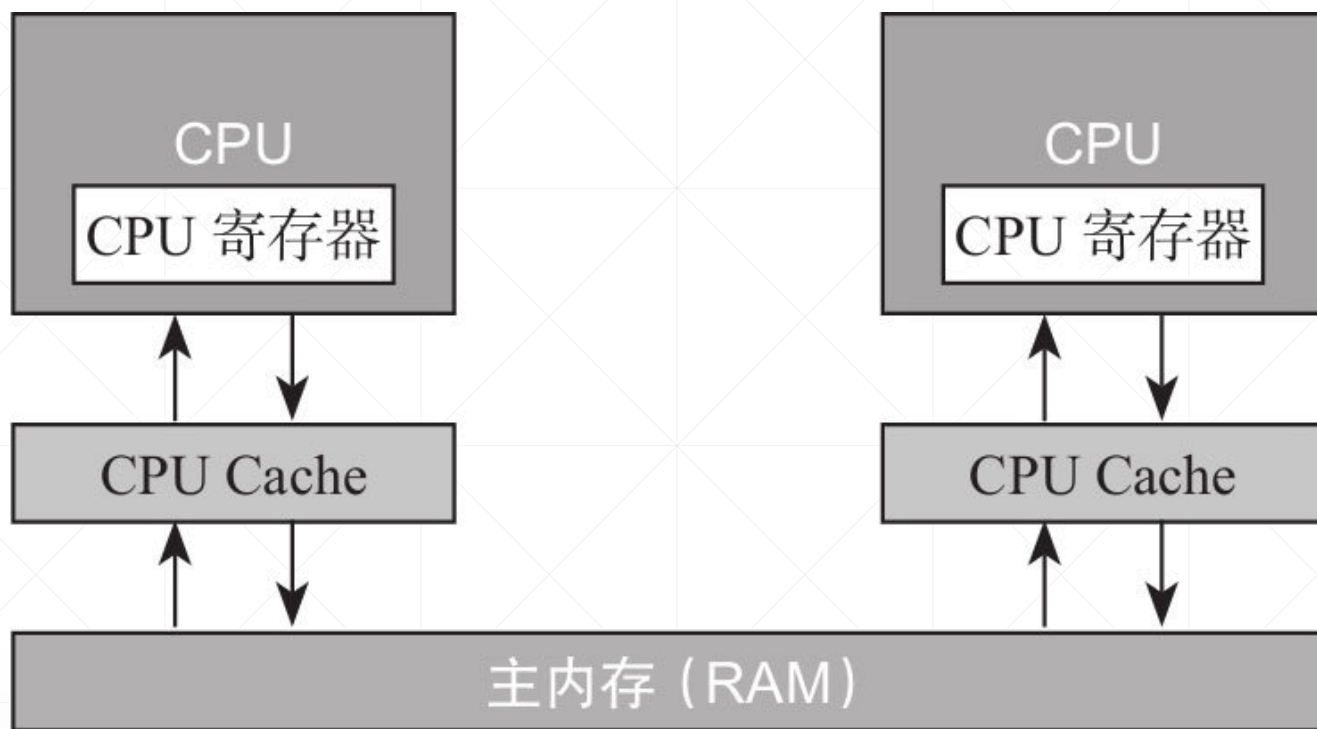


L1,L2,L3: 一级、二级、三级缓存

L1i: 一级指令 (instruction) 缓存

L1d: 一级数据 (data) 缓存

缓存中的数据，以Cache Line为存取单位。



Cache的出现是为了解决CPU直接访问内存效率低下问题的，程序在运行的过程中，会将运算所需要的数据从主存复制一份到CPU Cache中，这样CPU进行计算时就可以直接对CPU Cache中的数据进行读取和写入，当运算结束之后，再将CPU Cache中的最新数据刷新（flush）到主内存当中。

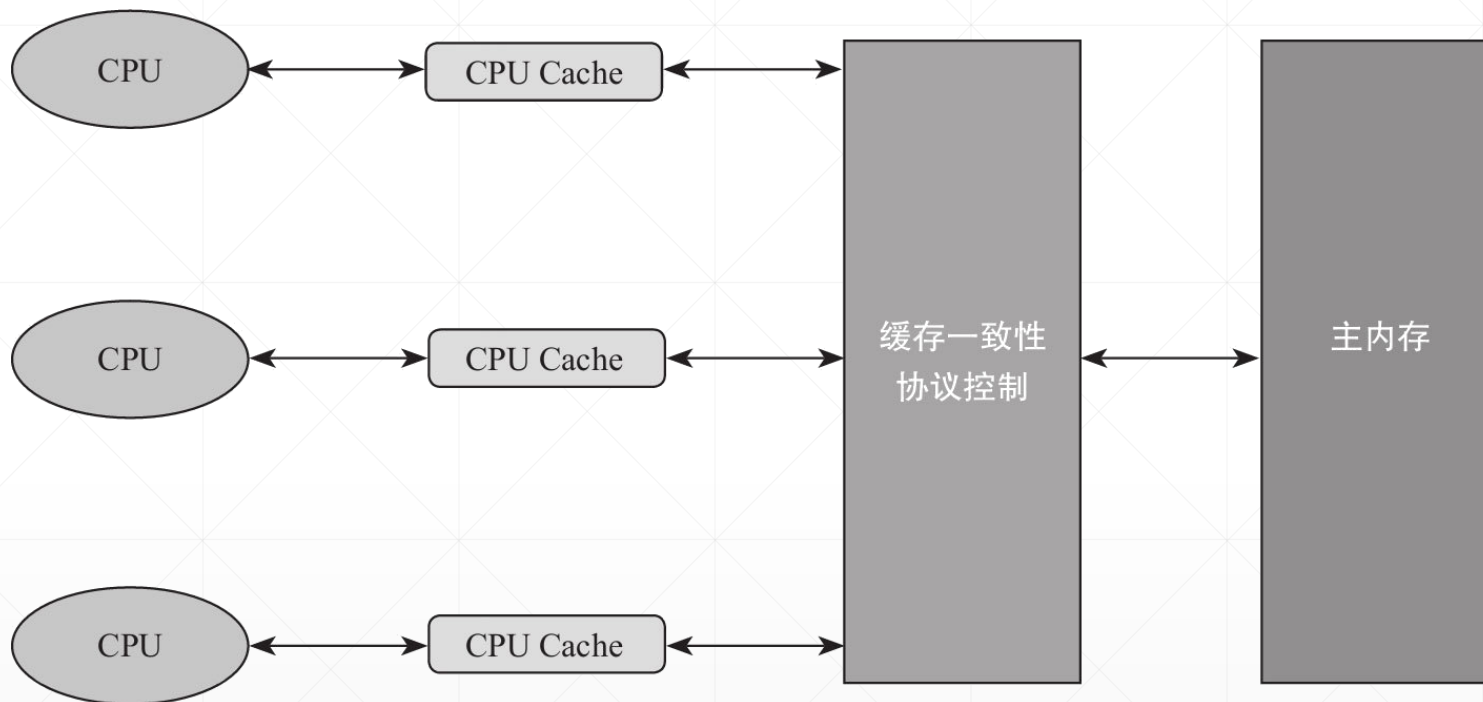
多线程环境下，多核CPU缓存一致性问题

举个例子，假设要计算 $i++$ 的值，这个数据操作可以分解为以下四步：

- 1) 读取主内存中变量 i 的值到CPU Cache中。
- 2) 对 i 进行加一操作。
- 3) 将结果写回到CPU Cache中。
- 4) 将数据刷新到主内存中。

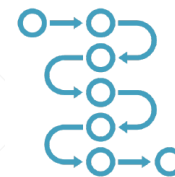
多线程环境下，如果多个线程都需要访问变量 i ，这就意味着变量 i 会在多个线程的本地内存中都存在一个副本，当一个线程修改了 i 值时，这种“变化”必须“传播到”主存和所有线程的本地内存（CPU缓存）中，否则，就有可能出现数据不一致的问题，导致数据处理出错。

缓存一致性协议



缓存一致性，是多线程程序设计与开发过程中，必须认真考虑的问题。

指令重排序



现代计算机系统结构设计的一个重要原则，就是尽可能地减少CPU的等待与闲置时间，让它高效率地运转。



为了达到这个目标，人们采取了许多方法，其中一个很重要的手段，就是在保证程序正确性的前提下，对要执行的指令进行重新排序，以便充分利用现代CPU的计算能力，实现高性能计算。



另一方面，现代编译器也会在编译时对原有代码进行优化，适当地调整生成代码指令的执行顺序，以便实现更高的并行。



指令重排序主要是由计算机系统硬件以及编译器来完成的，对程序员来说不可控，但由于这种重排序的存在，可能会对多线程程序的正确性带来影响，因此，不能对之“置之不理”。

内存系统重排序



由于CPU的运行速度与内存存取速度的不一致，现代CPU普遍使用了高速缓存。



当CPU需要将某数据写入主内存时，它不会等待这个数据存取操作完成才执行下一条指令，而是将数据写入高速缓存后就立即返回，之后再吧数据从缓存同步更新到主内存。

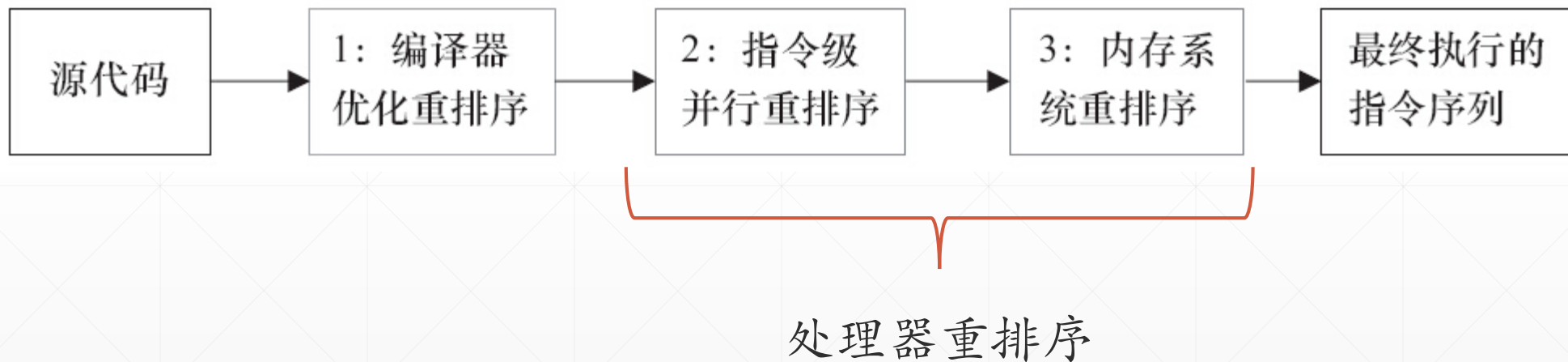


由于CPU保存数据时“立即返回”“不等待”，数据保存开始与真正结束之间存在着“时间差”，这就使得指令中规定的数据操作顺序，与真正执行的数据操作顺序之间不一致，这种现象，称为“内存系统重排序”。

小结：多线程程序的重排序



由于CPU是多核的，并且普遍拥有多级缓存，因此，存在着两个重要的技术特性——多核CPU数据缓存一致性和重排序的问题。



数据缓存一致性和重排序，都可能会对多线程程序的正确性有所影响，编写多线程代码时，必须高度地小心。